34

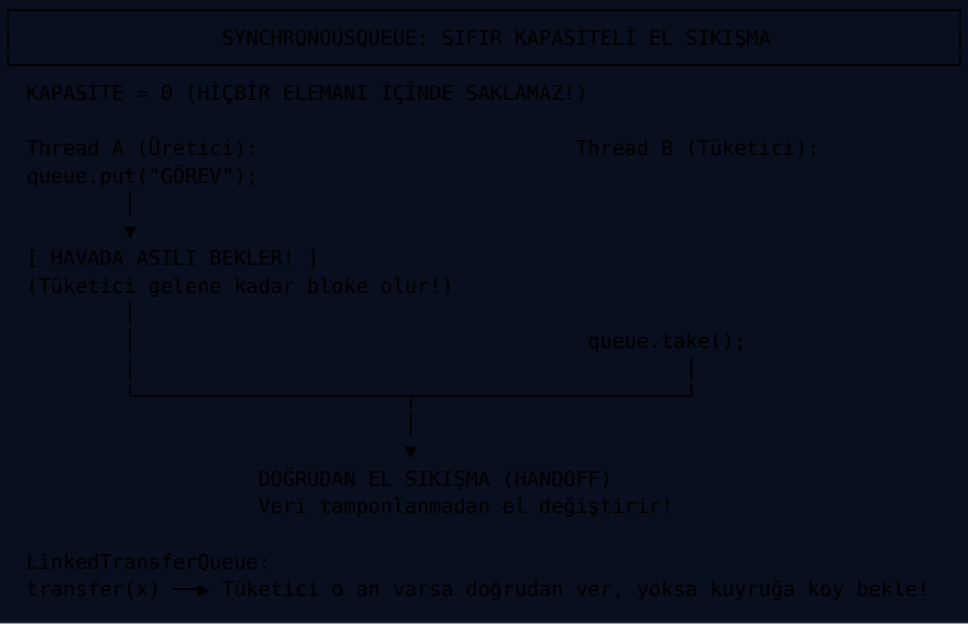
BÖLÜM: EŞZAMANLI KOLEKSIYONLAR & THREAD-SAFE VERI YAPILARI | MODULE: CONCURRENT COLLECTIONS & THREAD-SAFE DATA STRUCTURES

Sıfır Kapasiteli El Sıkışma: SynchronousQueue vs TransferQueue

Zero-Capacity Direct Handoffs: SynchronousQueue vs TransferQueue

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normal bir restoranda aşçı yemeği yapar, tezgahın üzerine koyar (Tamponlu kuyruk). SynchronousQueue'da ise tezgah YOKTUR (Kapasite = 0)! Aşçı sıcak tabağı elinde havada tutar (`put()`). Bir garson gelip tabağı aşçının elinden alana kadar (`take()`) aşçı kaskatı bekler! Tabak elden ele doğrudan teslim edilir. Arada hiçbir bekleme masası yoktur. Eğer hemen teslim alamayacaksan, aşçı yemeği pişirmeye bile kalkışmaz!

🇬🇧 ENGLISH (ELI5)

In a standard cafeteria, chefs set plates onto a warming counter (Buffering). A SynchronousQueue HAS NO COUNTER (Capacity = 0). The chef holds the scalding plate in mid-air (`put()`). The chef stands frozen in place until a server arrives to take the plate directly from their hands (`take()`). The transfer is an immediate atomic handoff from one thread's registers to another. If no server is waiting, the producer blocks instantly.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Doğrudan El Sıkışma (Direct Handoff) ve Dual Stack/Queue Mimarisi: 1) Sıfır Kapasite Felsefesi: `SynchronousQueue.size()` daima 0 döner; `peek()` daima null döner. Bir elemanı kuyruğa koymak ile o elemanın başka bir thread tarafından teslim alınması aynı zaman diliminde gerçekleşmek zorundadır. 2) Non-blocking Dual Queue (Scherer-Scott Algoritması): SynchronousQueue içsel olarak kilit kullanmaz; bekleyen üreticileri veya tüketicileri temsil eden düğümleri (QNode) lock-free bir yığında (TransferStack) veya kuyrukta (TransferQueue) CAS ile yönetir. 3) LinkedTransferQueue: Java 7 ile gelen bu yapı hem `BlockingQueue` hem de `TransferQueue` özelliklerini birleştirir. `tryTransfer(e)` metodu eğer bekleyen tüketici varsa anında el sıkışır, yoksa beklemeden false döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Zero-Capacity Handoff Mechanics: 1) The Non-Buffering Invariant: `SynchronousQueue` has zero internal storage. `put(e)` blocks until a concurrent `take()` arrives; `take()` blocks until a concurrent `put(e)` arrives. 2) Scherer-Scott Dual Stack/Queue: Implemented via lock-free dual data structures. Nodes represent either data requests or data fulfillment. Threads match against opposite-mode nodes using atomic CAS operations, eliminating monitor overhead. 3) `LinkedTransferQueue`: Blends `LinkedBlockingQueue` buffering with `SynchronousQueue` handoffs. The `transfer()` method blocks until a consumer accepts the item; `tryTransfer()` offers instant zero-wait handoffs.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: SynchronousQueue ile Doğrudan Handoff Gösterimi
import java.util.concurrent.SynchronousQueue;

public class DirectHandoffDemo {
    void main() throws InterruptedException {
        var syncQueue = new SynchronousQueue<String>();

        // Tüketici Thread (1 saniye geç başlıyor)
        Thread.ofPlatform().name("Consumer").start(() -> {
            try {
                Thread.sleep(1000);
                IO.println("Tüketici geldi, veriyi elden alıyor...");
                String item = syncQueue.take();
                IO.println("Tüketici teslim aldı: " + item);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Üretici Thread
        IO.println("Üretici: Veri teslim edilmeye çalışılıyor (put)...");
        long start = System.currentTimeMillis();
        syncQueue.put("Ultra-Önemli VIP Mesaj"); // Tüketici gelene kadar 1 saniye burada BLOKE OLUR!
        long elapsed = System.currentTimeMillis() - start;

        IO.println("Üretici: Veri el sıkışarak teslim edildi! Geçen süre: " + elapsed + " ms");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Executors.newCachedThreadPool() hangi kuyruğu kullanır ve neden?' Yanıt: 'SynchronousQueue kullanır! Çünkü gelen görevi hiçbir ara tamponda bekletmeden hemen boşta bekleyen bir thread'e teslim etmek ister; boşta thread yoksa anında yeni bir thread açar!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Crucial Interview Question: 'Which queue powers `Executors.newCachedThreadPool()` and why?' Answer: 'It uses `SynchronousQueue`! It demands immediate direct handoff to an idle worker. If all workers are busy, the handoff fails immediately, forcing the pool to spawn a new native thread instantly.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- SynchronousQueue'nun kapasitesi sıfırdır; veriyi asla bellekte tutmaz.
- put() çağrısı başka bir thread take() çağırana kadar bloke bekler.
- Executors.newCachedThreadPool doğrudan handoff için SynchronousQueue kullanır.

🎯 Key Takeaways for the Interview

- SynchronousQueue has zero internal capacity and never buffers elements.
- put() blocks until a matching take() arrives, executing an atomic thread-to-thread handoff.
- newCachedThreadPool relies on SynchronousQueue to enforce immediate task assignment.